

Data Analytics

Spatial Data Analytics: Fortsetzung

Prof. Stefan Keller, FS26

Inhalt VoWo 04 - Spatial Data Analytics: Fortsetzung

- Events
- VoWo 03:
 - Repetition
 - Ergänzung Kapitel "Geodaten-Formate" (Folie 52 ff.)
- PostGIS, die PostgreSQL-Erweiterung (Datenbank, Schema/Tabellen)
- (PostGIS-)Funktionen auf Vektor-Datentypen (inkl. Indexe)
- Verfügbare Datenquellen für Geodaten
- Tipps und Diskussion

Repetition Spatial Data Analytics: Einführung

- Geodaten: Daten mit Raumbezug: z.B. Koordinaten aber auch Geonamen
- Koordinatenreferenzsysteme
- Metadaten
- Ebenenprinzip ("Layers")
- Modellierung: Entitäten-/Objekte-Sicht vs. Felder-Sicht
- Datenstrukturierung: Vektor- und Raster-Modell (und Grid, Topologie)
- Massstabsabhängige Darstellung ("GIS-Zoom")
- Mengenbeschränkte Darstellung, Generalisierung, Priorisierung

GEO-DATENTYPEN: STRUKTUREN, OPERATIONEN, INDEXE

- Vektor-Datentypen
 - z.B. mit GeoJSON oder andere Binärformate
 - (unser Schwerpunkt hier!)
- Raster-Datentypen
 - Datenstruktur Pixel, z.B. GeoTIFF
 - Datenstruktur regelmässiges "Grid"
 - GeoTIFF mit R (von RGB) als Wert(e)
 - Aber auch als regelmässige Grid-Vektor-Punkte z.B. als CSV mit Koordinaten
- Topologie-Datentypen
 - "Nachbarschafts-Beziehungen" von Knoten und Kanten, "codiert" als
 - 1. Vektor-LineStrings mit gemeinsamen Knotenpunkte (z.B. OpenStreetMap)
 - 2. spezialisierte Datenstruktur, z.B. Von-/Bis-Referenzen auf Endknoten(-punkte)

VEKTOR-DATENTYPEN: BEZUG ZUR GEOMETRIE

- Basisdatentypen (Vektor)
 - Point ("Punkt", Koordinate)
 - LineString/Polylinie ("Linie")
 - Polygon ("Fläche")
- Punkte: Probleme schon bei diesem Datentyp (2D):
 - lat/lon oder lon/lat?
 - Evtl. die Höhe "Z-Koordinate" (=2.5D)? Echte 3D-Objekte (Polyhedron), M-Wert
- Linien: Definition "Was ist eine Linie?" ist bereits ein Challenge
 - Genau 2 Punkte, Anfangs- und Endpunkt? Mehrere Stützpunkte?
 - Definitions-Versuch: "Eine geordnete Folge von Stützpunkten, die durch Linien verbunden sind, entweder linear (default) oder als Kreisbogen, Spline, NURBS..."

Realisierung Geometrie-Datentyp

- Sequentielle Dateien
- Normalisierte Tabellen
- Binary Large Objekts (BLOB)
- und (schliesslich) User Defined Type (UDT, gemäss SQL:99)
 - Binary Large Objekts (CLOB/BLOB) gekapselt über
 - Konstruktoren
 - Funktionen
 - REF-Typen

Datentypen: Vektorgeometrie

- GEOMETRY (abstrakter Obertyp)
 - POINT, LINESTRING, POLYGON,
 - MULTIPOINT, MULTILINESTRING, MULTIPOLYGON
 - GEOMETRYCOLLECTION

Gegeben eine Tabelle "ort"

```
CREATE TABLE ort(  
  id INT PRIMARY KEY,  
  name TEXT,  
  geom GEOMETRY(POINT, 4326)  
);
```

Fügen wir eine Zeile ein mit einem GEOMETRIE-Konstruktor (OGC, ISO 19100, SQL/MM):

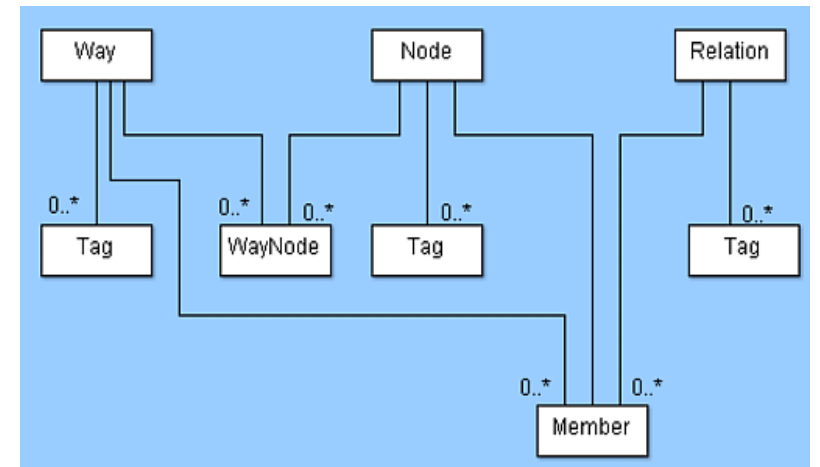
```
INSERT INTO ort(id, name, geom)  
VALUES (  
  100,  
  'Rapperswil (SG)',  
  ST_GeometryFromText('POINT(8.818 47.226)', 4326) -- Well Known Text  
);
```

Hinweis: Rapperswil liegt bei "47.226, 8.818" lat/lon, vgl. z.B. GeoAdmin, OSM oder GMaps

DATENQUELLE OPENSTREETMAP (OSM)

Topologie-Datentyp: Beispiel Datenmodell OpenStreetMap (OSM)

- Attribute als "Tags" (= Key/Value-Paare)
- Konzeptionelles Schema: Point, Line, Area (GIS/Geometrie-Datentypen)
- Physisches Schema: Nodes, Ways, Relations (OSM-Elemente)
 - Nur Nodes enthalten Koordinaten
 - Ein Way enthält Array von Nodes
 - Flächen, kein eigenständiger Typ:
 - geschlossene Ways
 - oder Relation au Ways als Ränder
 - (Achtung Multipolygon: Leicht andere Def. als WKT!)
 - Nodes, Ways und Relations...
 - können 0, 1 bis mehrere Tags haben
 - Gemeinsame Attribute: `osm_id`, `user_id`, `last_modified`, `version`, `changeset`, ...



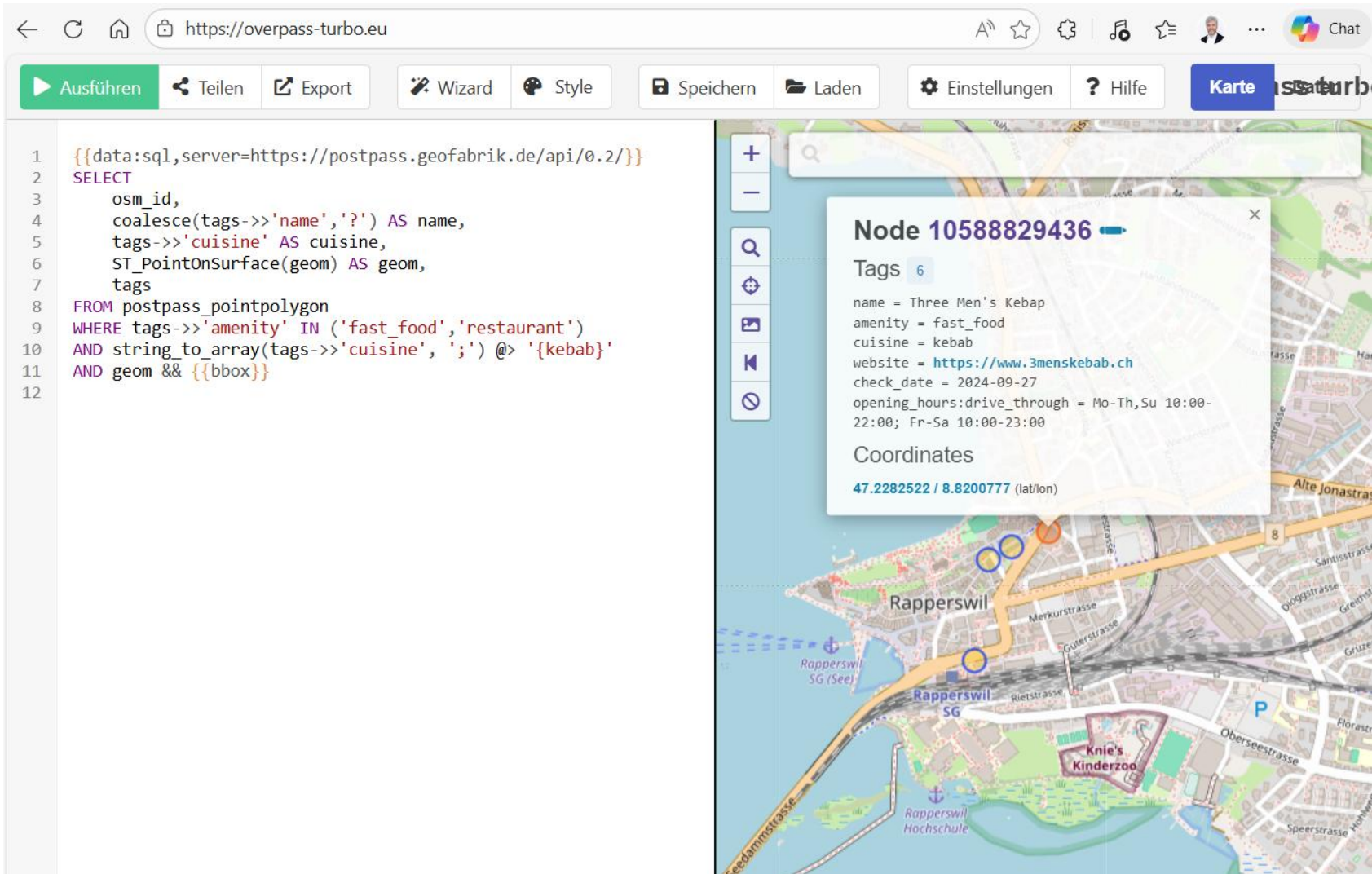
- Overpass API mit mit Overpass-Turbo
 - Weltweit, Schema nahe an OSM-Elementen
 - Kryptische Abfragesprache Overpass QL (die XML-Variante ist deprecated)
 - Ok bis Abfragen der Grösse eines Quartiers/Stadtviertels
 - Verschiedene Overpass-Server
 - Python-Bibliotheken OSMNx und Webapps bauen darauf
- PostPASS mit Overpass-Turbo
 - SQL-Frontend zu einer PostGIS-Datenbank
 - Weltweit, Schema näher an Geoinformationssysteme mit Point, LineString, Polygon
 - Siehe →

- OpenStreetMap-Daten weltweit(!), stündlich aktualisiert
- Schema:
 - `postpass_point` - alle OSM-Punkte (Nodes mit Tags).
 - `postpass_line` - Linien/Wege (Ways, die als Linie repräsentiert werden, z.B. Strassen, Railway).
 - `postpass_polygon` - Polygone (geschlossene Ways oder areal repräsentierte Flächen).
 - `postpass_pointpolygon` - eine View, die zu Point auch Polygone (mit UNION) mischt, so dass man mit `ST_PointOnSurface` die Polygone-Punkte dazubekommt
 - Weitere Tabellen
- Diese Tabellen haben folgende gemeinsame Attribute:
 - `osm_type` & `osm_id` – Der OSM-interne Identifikator, z.B. node & 12397780545
 - `tags` vom Typ json
 - `geom` vom Typ GEOMETRY mit EPSG 4326 (WGS84)
- Quelle: <https://github.com/woodpeck/postpass-ops/blob/main/SCHEMA.md>

Zurzeit operativ nur auf dieser Instanz: <https://overpass-turbo.eu/>

```
{{data:sql,server=https://postpass.geofabrik.de/api/0.2/}}  
SELECT  
    osm_id,  
    coalesce(tags->>'name','?') AS name,  
    tags->>'cuisine' AS cuisine,  
    ST_PointOnSurface(geom) AS geom,  
    tags  
FROM postpass_pointpolygon  
WHERE tags->>'amenity' IN ('fast_food','restaurant')  
AND string_to_array(tags->>'cuisine',';') @> '{kebab}'  
AND geom && {{bbox}}
```

PostPass - Webapp



The screenshot shows the PostPass web application interface. The browser address bar displays `https://overpass-turbo.eu`. The application has a top navigation bar with buttons: **Ausführen** (green), **Teilen**, **Export**, **Wizard**, **Style**, **Speichern**, **Laden**, **Einstellungen**, **Hilfe**, and **Karte** (blue). Below the navigation bar, the left panel contains a SQL query editor with the following code:

```
1  {{data:sql,server=https://postpass.geofabrik.de/api/0.2/}}
2  SELECT
3    osm_id,
4    coalesce(tags->>'name','?') AS name,
5    tags->>'cuisine' AS cuisine,
6    ST_PointOnSurface(geom) AS geom,
7    tags
8  FROM postpass_pointpolygon
9  WHERE tags->>'amenity' IN ('fast_food','restaurant')
10 AND string_to_array(tags->>'cuisine',';') @> '{kebab}'
11 AND geom && {{bbox}}
12
```

The right panel displays a map of Rapperswil, Switzerland. A popup window titled **Node 10588829436** is open, showing the following details:

- Tags** 6
 - name = Three Men's Kebab
 - amenity = fast_food
 - cuisine = kebab
 - website = <https://www.3menskebab.ch>
 - check_date = 2024-09-27
 - opening_hours:drive_through = Mo-Th,Su 10:00-22:00; Fr-Sa 10:00-23:00
- Coordinates**
[47.2282522 / 8.8200777](#) (lat/lon)

The map shows the town of Rapperswil, including the Rapperswil SG (See) area, the Rapperswil Hochschule, and the Knie's Kinderzoo. The map is overlaid with a grid of nodes and lines, representing the OpenStreetMap data.

VEKTOR-DATENTYPEN: FUNKTIONEN

- Bei raumbezogenen Daten/Geodaten interessieren nicht nur die geografische Lage...
- ...sondern auch Beziehungen (relationships) zwischen Objekten, die sonst aufwändig modelliert und erfasst werden müssten:
 - Elegant mit Geometrie-Attributen: Tabellen "Kantone" (polygon), "Berge" (point)
 - Aufwändig mit Beziehungen: Tabellen "Kantone" und "Berge" mit Attribut "gehört zu"
- Es geht um Topologie, die als Geometrie "codiert" ist
- Typische topologische Beziehungen sind
 - Nähe (proximity): Distanz
 - Nachbarschaft (adjacency): grenzen an, (touching), verbunden mit (connectivity)
 - Containment : inside/overlapping

Funkt./Raumanalyse: Klassierung (1)

- Raumanalyse-Funktionen/Operationen:
 - basieren auf konzeptionellen Modellen/Sichten, nicht Datenstrukturen!
 - erzeugen neue Daten → sekundäre/abgeleitete Daten
- Grobe Einteilung:
 - Geometrische, topologische, thematische Abfragen
- Goodchild (1990, „Spatial Analysis Using GIS“)
 - Zählen und Messen:
 - Zählen von Objekten in einem Teilgebiet
 - Berechnen der Distanz
 - Längenberechnung der Grenzlinie eines Polygons

(Fortsetzung nächste Folie...)

Funkt./Raumanalyse: Klassierung (2)

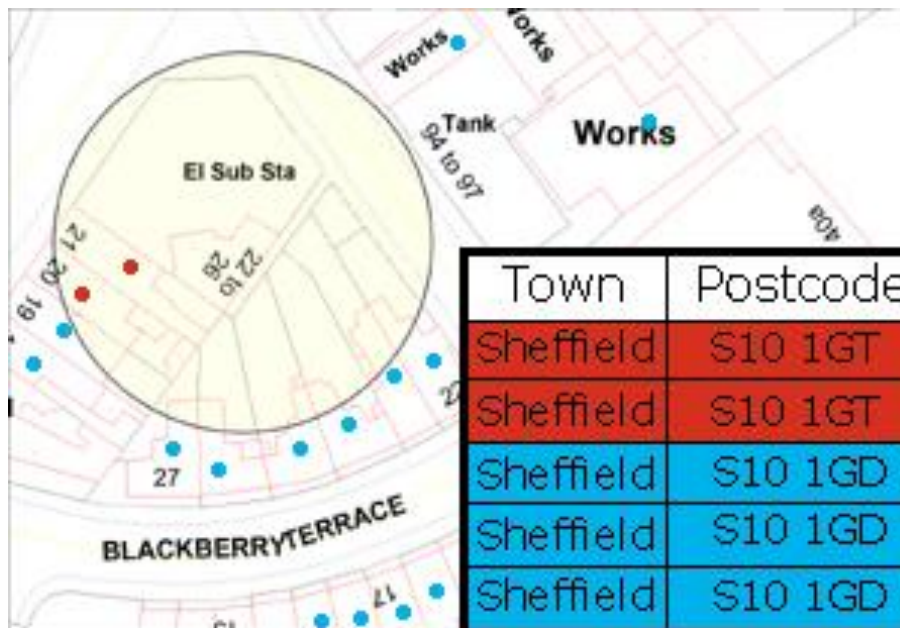
- Goodchild ff.
- Funktionen der Raumanalyse:
 - Topopol. Netzwerkanalyse (Routing z.B. Verkehr, Flüsse)
 - Überlagern von Punkt im Polygon (point-in-polygon test), oder von Linien/Polygonen mit anderen Linien/Polygonen (intersect/overlay)
- Geländemodellierung:
 - Interpolation der Höhe an jedem beliebigen Punkt
 - Berechnung von Höhenkurven aus einem unregelmässigen Punktnetz
 - Berechnen der mittleren Neigung von Flächen
- Statistische Analysefunktionen:
 - Arithmetische, algebraische und logische Operationen an Geometrie- und Attributdaten
 - Statistische Verfahren und Tests

Funkt./Raumanalyse: Klassierung (3)

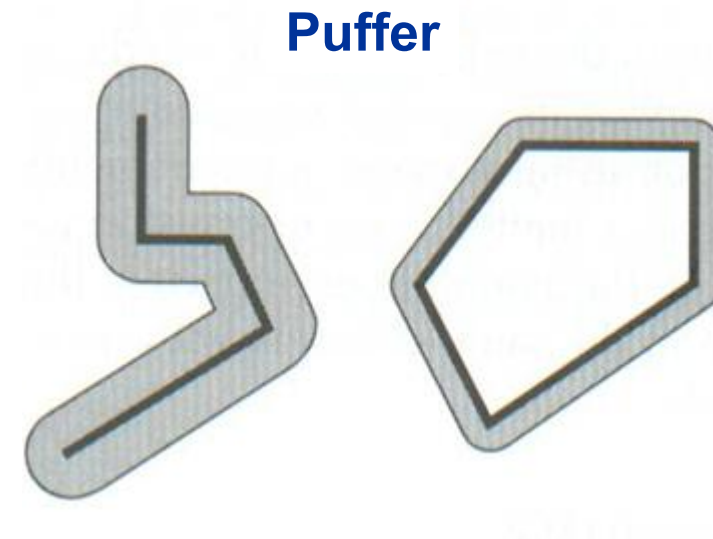
- Keller (2011): Klassen angelehnt an GIS-SW-Funktionen (Fn.)
- "g" und "g1" etc. seien Attribute vom Typ geometrie:
 - Management Functions (return geometries)
 - Editor, Accessor and Constructor Fns., z.B.
 - Output, Conversion and Helper Functions (return geometries)
 - z.B. st_asgeojson (geom), ...
 - Measurement Functions (return numbers)
 - z.B. st_length(geom), ...
 - Relationship Functions (return boolean)
 - z.B. st_intersects (g1,g2), st_dwithin(g1,g2), ...
 - Spatial Aggregate Functions (return scalar geometries)
 - z.B. st_union(g1,g2), ... Achtung "Grenzvereinigung"!
 - Processing Functions (return geometries)
 - z.B. st_intersection(g1,g2), st_buffer(g1,d), ...

Funktionen der Raumanalyse

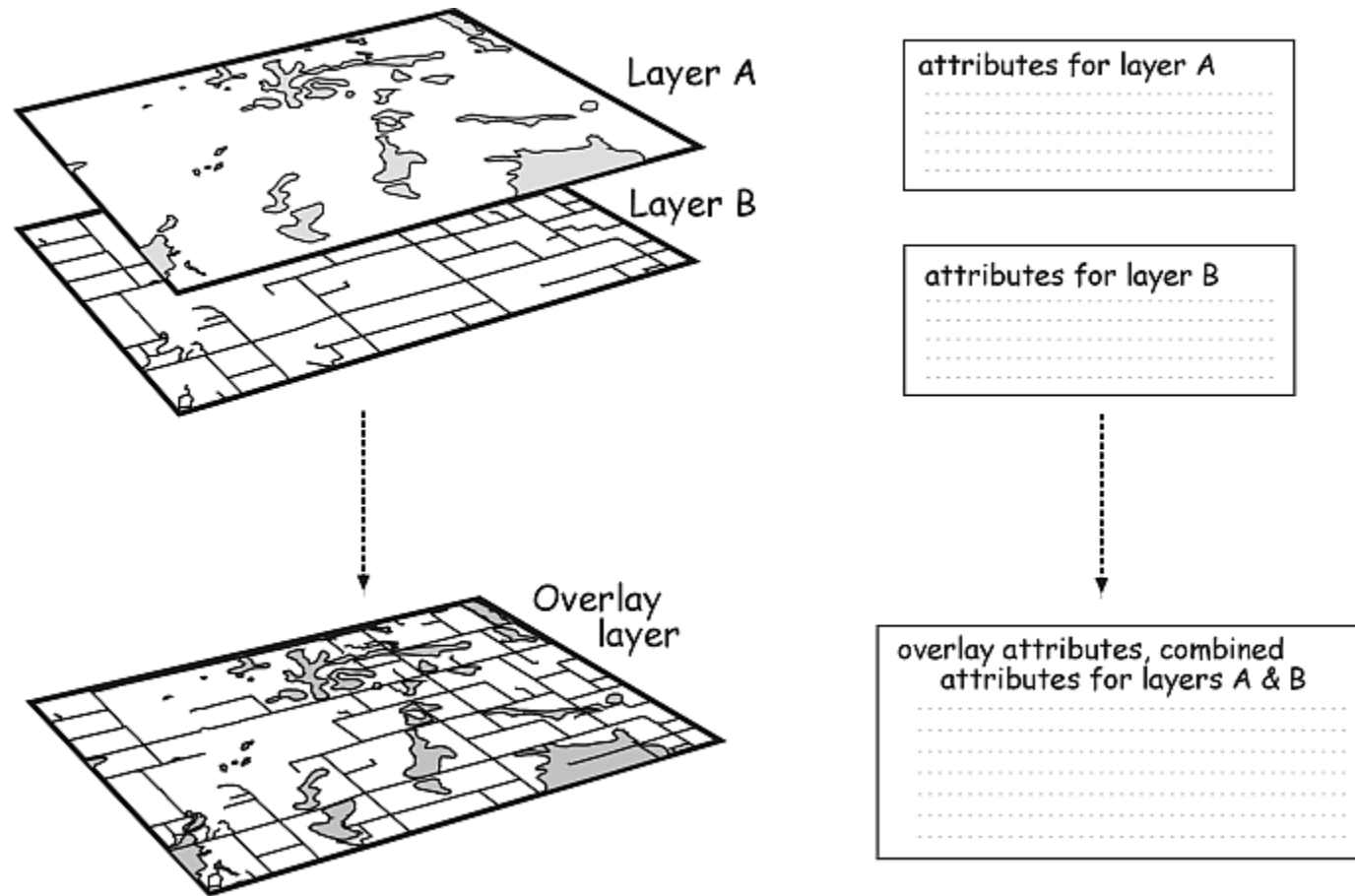
Point-in-Polygon



Puffer



Funktionen der Raumanalyse: Overlay / Intersect / Verschnitt mit Vektordaten



Funktionen der Raumanalyse: Overlay / Intersect / Verschnitt mit Rasterdaten

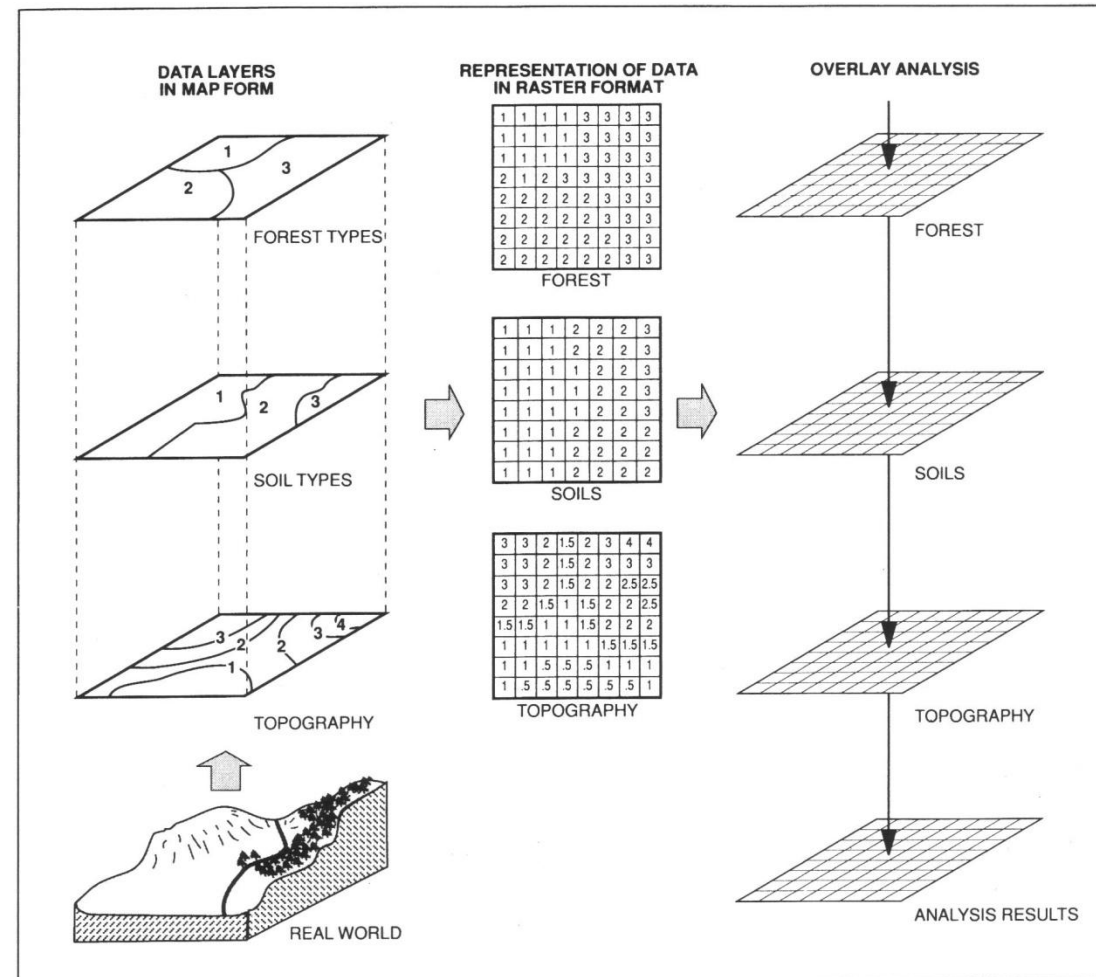


Figure 6.11 Overlay Analysis Using Raster Data Files.

GEO-NORMEN UND -STANDARDS

- Standardisierungsgremien:
 - W3C, ISO, CEN (EU), SNV / eCH (Schweiz)
 - "Industrienorm" Open Geospatial Consortium (OGC)
- Vektor:
 - Shapefile (de-facto) → veraltet: siehe <http://switchfromshapefile.org/> warum
 - GeoJSON (W3C), GeoPackage (Sqlite, OGC)
 - DXF (v.a. Geometrie), KML (XML, OGC)
 - Interlis (XML+XML Schema)
 - Web Feature Service (WFS) aka pseudo REST API mit XML (OGC)
 - "OGC API - Features" aka REST mit GeoJSON (OGC), Nachfolge-Standard von WFS
- Raster:
 - GeoTIFF
 - Web Map Service (WMS) aka pseudo REST API (OGC)
 - "OGC API - Maps", Nachfolge-Standard von WMS

VEKTOR-DATENTYPEN: POSTGRESQL / POSTGIS

- Erweiterung von PostgreSQL
 - Funktionen
 - Externe Bibliotheken (z.B. zur Koordinaten-Transformation)
 - Data Loader Tool
- Installation z.B. mit psql
 - CREATE EXTENSION PostGIS;
- Ausgereifteste Open Source-DB-Erweiterung:
 - Kompatibel zur OGC-Norm "Simple Features for SQL"
 - Hunderte von Geometrie-Funktionen, z.B. ST_Buffer()
 - Operatoren auf Geometrie-Datentypen
 - Spezieller Index auf PostgreSQL "GIST" als R-Baum (R-tree) = mehr-dimensionale, dynamische, balancierte Datenstruktur

Tabelle mit Geometrie-Datentyp

- mit Primärschlüssel
- und räumlichem Attribut ,geom'
 - Mit Type Modifier LINESTRING (analog VARCHAR(20))
 - Und Koordinatensystem (CRS, SRS, EPSG): 4326 (WGS84,GPS), 3857 (Web-Mercator), 2056 (CH1903+/LV95), 21781 (CH1903/LV03)

```
CREATE TABLE eisenbahn (  
  id int4 PRIMARY KEY,  
  name TEXT,  
  geom GEOMETRY ( ' LINESTRING ' , 2056)  
);
```

- Veraltet (nur verwenden wenn man weiss was man tut)!
`SELECT AddGeometryColumn (...)`

- PostGIS Polygon-Konstruktor mit extended WKT (Well Known Text):
POLYGON((0 0, 0 1, 1 1, 1 0, 0 0))



Geometry Type:

-- Simplest text form without SRID:

```
SELECT ST_AsEWKT(ST_GeomFromText('POINT(-71.06 42.28)'))
```

-- For text form with SRID:

```
SELECT ST_AsEWKT(ST_GeomFromText('POINT(-71.06 42.28)', 4326))
```

-- Symbolic form:

```
SELECT ST_AsEWKT(ST_SetSRID(ST_MakePoint(-71.06, 42.28), 4326))
```

```
SELECT ST_AsEWKT(ST_Point(-71.06, 42.28, 4326))
```

Geography Type:

```
SELECT ST_AsText(ST_GeogFromText('SRID=4326;POINT(-71.06  
42.28)'))
```

-- (Hint: ST_AsEWKT doesn't work)

– Siehe auch <https://www.giswiki.ch/PostGIS>

Tabelle mit Geometrie-Datentyp ff: Index

- Dann räumlichen Index erstellen:

```
CREATE INDEX  
  in_eisenbahnen_the_geom  
ON eisenbahnen  
USING gist (geom) ;
```

- Optional: Weitere Indizes erstellen (wie gehabt...)

Tabelle laden und testen (WKT)

```
INSERT INTO uster(id, name, geom)
VALUES (
    100,
    'Uster'
    ST_GeometryFromText('
        LINESTRING(8.81 47.22,8.82 47.23)',4326)
);
```

```
SELECT id, name, ST_AsText(geom)
FROM uster;
id | name | geom
-----
100    Uster  LINESTRING(8.81 47.22, 8.82 47.23)
1 row;
```


VEKTOR-DATENTYP: POSTGIS-FUNKTIONEN

PostGIS Funktionen (>330) in SQL

- Constructors & Helpers:
 - text ST_AsText(geom g), ST_GeomFromText('...'), ST_MakePoint()
 - geom ST_Transform(geom g, integer srid)
 - float ST_Distance(geom g1, geom g2) – in CRS Units
- Filters:
 - boolean ST_Intersects(geom g1, geom g2) – Es gibt noch ST_Contains, ST_Within und ST_Covers, wobei normalerweise ST_Intersects vorzuziehen ist.
 - boolean ST_DWithin(geom g1, geom g2, distance d)
- Analysis:
 - geom ST_Buffer(geom g, float radius_of_buffer)
 - geom ST_Intersection(geom g1, geom g2)
 - geom ST_Union(geom g1)

PostGIS Funktionen (>330) in SQL: Cheatsheet

Management		Accessors
AddGeometryColumn DropGeometryColumn DropGeometryTable <u>populate geometry columns</u> ³ postgis_full_version postgis_geos_version postgis_lib_version postgis_proj_version postgis_version probe_geometry_columns ST_SetSRID UpdateGeometrySRID	This list is not comprehensive but tries to cover at least 80%. *Uses GEOS, Requires Geos 3.2+ to take advantage of new or improved features^{G3.2} Most commonly used functions and operators Measurement functions return in same units geometry SRID except for the *sphere and *spheroid versions and new Geography type which return in meters Denotes a new item in this version ¹ Denotes automatically uses spatial indexes ² Denotes enhanced in this version ³	ST_CollectionExtract ¹ ST_Dimension ST_Dump ST_DumpPoints ¹ ST_DumpRings ST_EndPoint ST_Envelope ST_ExteriorRing ST_GeometryN ST_GeometryType ST_InteriorRingN ST_IsClosed ST_IsEmpty ST_IsRing ST_IsSimple ST_IsValid ST_IsValidReason ST_mem_size ST_M ST_NumGeometries ST_NumInteriorRings ST_NumPoints ST_npoints ST_PointN ST_SetSRID ST_StartPoint ST_Summary ST_X ST_XMin, ST_XMax ST_Y YMin, YMax ST_Z ZMin, ZMax
Load/Dump Tools		
--PostGIS tools -- shp2pgsql shp2pgsql-gui ³ pgsql2shp --PostgreSQL -- pg_dump pg_restore psql	GEOMETRY/GEOGRAPHY TYPES - WKT REPRESENTATION POINT(0 0) LINESTRING(0 0,1 1,1 2) POLYGON((0 0,4 0,4 4,0 4,0 0),(1 1, 2 1, 2 2, 1 2,1 1)) MULTIPOINT(0 0,1 2) MULTILINESTRING((0 0,1 1,1 2),(2 3,3 2,5 4)) MULTIPOLYGON(((0 0,4 0,4 4,0 4,0 0),(1 1,2 1,2 2,1 2,1 1)), ..) GEOMETRYCOLLECTION(POINT(2 3),LINESTRING((2 3,3 4))) BBOX AND GEOMETRY OPERATORS A << B (A overlaps or is to the left of B) ² A <> B (A overlaps or is to the right of B) ² A << B (A is strictly to the left of B) ² A >> B (A is strictly to the right of B) ² A < B (A overlaps B or is below B) ² A > B (A overlaps or is above B) ² A << B (A strictly below B) ² A >> B (A strictly above B) ² A = B (A bbox same as B bbox) A @ B (A completely contained by B) ² A ~ B (A completely contains B) ² A && B (A and B bboxes intersect) ² A ~= B - true if A and B boxes are equal ^{2 3} COMMON USE SFSQL EXAMPLES --Create a geometry column named geom in a --table called testtable located in schema public -- to hold point geometries of dimension 2 in WGS84 longlat	Measurement ST_Area³ ST_Azimuth ST_Distance <u>ST_HausdorffDistance</u>^{1G3.2}
Meta Tables/Views		
spatial_ref_sys geometry_columns geography_columns ¹		
Geometry Creation		
ST_GeomFromEWKB ST_GeomFromEWKT ST_GeogFromText ¹ ST_GeomFromGML ¹ ST_GeomFromKML ¹ ST_GeomFromText ST_GeomFromWKB ST_GeogFromWKB ¹ ST_MakeEnvelope ¹		

Metrische Funktionen

- Rückgabe: Zahl
 - Länge
 - ST_Length(geom)
 - Abstand
 - Beispiel: Maximale Distanz eines Studenten zu seiner zugewiesenen Schule:

```
SELECT max(ST_Distance(  
    ST_Transform(s.geom, 2056), -- nach CH für Meter  
    ST_Transform(sc.geom, 2056)  
))  
FROM students s  
JOIN schools sc ON s.school_id = sc.id; Flächeninhalt
```
 - ST_Area(geom)

Geometrische Funktionen

- Rückgabe: Geometrie
- Beispiele:
 - Minimal umgebendes Rechteck
 - `ST_Envelope(geom)`
 - Pufferzone
 - `ST_Buffer(geom, distance)`
 - Konvexe Hülle
 - `ST_ConvexHull(geom)`
 - ...
- Man beachte: Können unerwartete Geometrien zurückgeben!

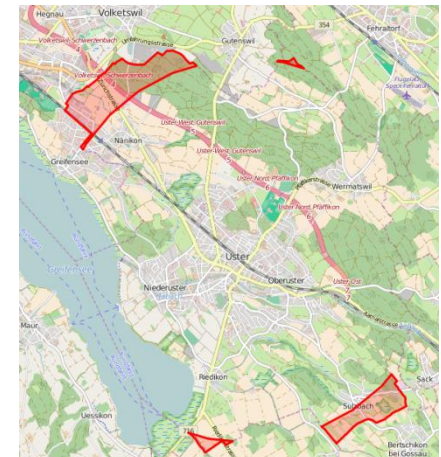
Verschneidung

Rückgabe: Geometrie

- ST_Intersection(g1 geom, g2 geom) -- Wichtige Funktion!
- ST_Union(g1 geom, g2 geom)
- ST_Difference(g1 geom, g2 geom)
- ST_SymDifference(g1 geom, g2 geom)

Beispiel: Alle Teilflächen von Uster, die >3000 m vom Mittelpunkt von Uster entfernt sind (OSM SQL Terminal):

```
SELECT
  label,
  ST_Transform( -- Transform back to EPSG:4326 after the operation
    ST_Difference(
      ST_Transform(geom, 2056),
      ST_Buffer(ST_Transform(ST_Centroid(geom), 2056), 3000) -- Buffer in meters
    ),
    4326
  )::geometry AS geom -- Ensure result is cast back to geometry
FROM osm_boundary
WHERE name LIKE 'Uster';
```



Test (Filter) topologischer Beziehungen

Aufbau:

```
OpName (  
    g1    GEOMETRY,    -- 1. Geometrie-Attribut  
    g2    GEOMETRY      -- 2. Geometrie-Attribut  
) RETURN BOOLEAN;    -- true falls Beziehung gilt
```

Auszug:

- ST_Equals(g1, g2): g1 ist gleich g2
- ST_Intersects(g1, g2): g1 und g2 haben eine Schnittmenge
 - (Gegenstück ST_Disjoint),
 - oft auch zur Indexierung verwendet mit &&-Operator und BBOX
 - Meist bevorzugt gegenüber ST_Within, ST_Inside, ST_Contains, ST_Covers!
- ST_Touches(g1, g2): g1 berührt Rand von g2 - ST_Disjoint: g1 berührt nicht g2
- ST_Crosses(g1, g2): g1 schneidet g2

VERFÜGBARE DATENQUELLEN

Tipps zu offenen Geo-Datenquellen

- "Raumbezug - der universelle Fremdschlüssel" ermöglicht die Kombination mit weiteren (offenen) Daten
- Open Government Data Schweiz
 - GeoAdmin map.geo.admin.ch (Swisstopo), SWISSGEO <https://test.swissgeo.ch>
 - opendata.swiss
 - Bundesamt für Statistik
 - GeoHarvester.ch
- Open Data
 - OpenStreetMap: osm.org bzw. "Overpass Turbo"; siehe auch www.openschoolmaps.ch
 - NASA (z.B. SRTM Höhenmodell)
 - Google Dataset Search <https://datasetsearch.research.google.com/>
 - Datensammlung [https://www.giswiki.ch/Freie Geodaten](https://www.giswiki.ch/Freie_Geodaten)
- Free Data und API (oft Freemium API)
 - Google Maps API
 - OpenMapTiles API

Quelle: Pat Browne, 2009-10

VEKTOR-GEODATENANALYSE: BEISPIEL „RAPPI BEERS“

Beispiel Rappi Beers:

- Erzeuge Tabelle "beers":

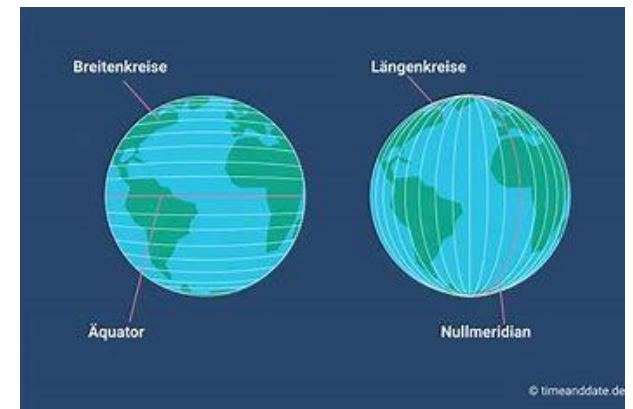
```
CREATE TABLE beers (  
  id GENERATED ALWAYS AS IDENTITY,  
  name VARCHAR(60),  
  price NUMERIC(10, 2),  
  geom GEOMETRY('POINT', 4326) -- geocentric  
  -- Ein planares CRS würde vieles vereinfachen:  
  -- GEOMETRY('POINT', 2057) -- planar for CH  
);
```

Rappi Beers: Daten einfügen

- Die Koordinaten 8.81915 / 47.22640 vom Manor mitten in der Stadt Rapperswil
- Zur Erinnerung: Die Zahlen sind Längengrad/Breitengrad (=Longitude/Latitude)



```
INSERT INTO beers (name, price, geom)
VALUES (
    'Manor',
    1.15,
    'POINT (8.81915 47.22640)'
);
```



Rappi Beers: Query 1 - by price

```
select * from beers order by price;
```

id	name	price	geom
10;	"Nelson Pub";	4.50;	"01010000..."
15;	"Bier Factory ";	3.90;	"01010000..."
14;	"Gourmellino";	2.40;	"01010000..."
13;	"Spar mini";	2.00;	"01010000..."
12;	"Coop Bahnhof";	1.80;	"01010000..."
11;	"Manor";	1.15;	"01010000..."
16;	"Coop Sunnehof";	1.05;	"01010000..."



Rappi Beers: Query 2 – by distance

```
select name, price,  
       round(  
         ST_DistanceSphere(  -- Note: Helper fn. to deal with lat/lon  
           geom,  
           ST_GeometryFromText('POINT(8.816511 47.223064)', 4326)  
         )::numeric, 0  
       ) as "distance"  
from beers  
order by distance;
```

"Nelson Pub";	4.50;	260
"Coop Bahnhof";	1.80;	285
"Gourmellino";	2.40;	300
"Manor";	1.15;	422
"Coop Sunnehof";	1.05;	706

Rappi Beers: Query 3 – the optimum!

```
select name, price,  
       round((price + 0.001 * ST_DistanceSphere(geom,  
         ST_GeometryFromText(  
           'POINT(8.816511 47.223064)', 4326)))::numeric, 2  
       ) as net_price  
from beers  
order by net_price;
```

"Manor";	1.15;	1.57
"Coop Sunnehof";	1.05;	1.76
"Coop Bahnhof";	1.80;	2.09
"Gourmellino";	2.40;	2.70
"Spar mini";	2.00;	2.91
"Nelson Pub";	4.50;	4.67
"Bier Factory";	3.90;	5.20

Zusammenfassung dieser Doppelstunde

- Sie kennen ausgewählte (PostGIS-)Funktionen darauf
- Sie kennen einige Prinzipien der Geoinformationsverarbeitung und der Spatial Data Analytics
- Sie kennen einige freie Datenquellen und Webservices
- Man merke zu Koordinatenreferenzsystemen:
 - Wichtig sind EPSG:4326 (lat/lon), EPSG:3857 (Mercator), EPSG:2056 (CH)
 - Man sage nicht X/Y sondern Ostwert "Easting" und Nordwert "Northing"
 - Man sagt oft "lat/lon" was Latitude/Longitude, bzw. Breitengrad/Längengrad ist
 - ... während der Standard und PostGIS-Funktionen "lon/lat" verlangen z.B. 8.81/47.22 für Rapperswil-Zentrum
 - Merksatz: Länge lang, Breite breit: Längengrade sind lang (vom Nord- zum Südpol), Breitengrade breit (waagerecht quer)

Ausblick

- Übungen: Mit PostGIS...
- Nächste VoWo 05: Big Data Analytics und Cloud Platforms